ACADEX AI

Agentic Academic Intelligence & Intervention System

Foundation of Agentic AI - Technical documentation

1. Abstract

Acadex AI is an agentic AI proof of concept for academic monitoring and early intervention. It demonstrates how synthetic student records can be explored through an interactive dashboard, processed through a coordinated multi-agent workflow, and converted into structured risk assessments and faculty-reviewed recommendations. The prototype combines Streamlit, FastAPI, Python-based rules, and Google Gemini-supported agent tasks.

2. Problem Statement and Domain Justification

Educational institutions may need to review attendance, assessment performance, and assignment completion across many students. Manual review can make it difficult to identify concerning patterns consistently and to prepare timely follow-up. Acadex AI explores an education-domain decision-support workflow that brings these indicators together and helps faculty prioritize review. It is a prototype, not an automated decision-maker.

3. Proposed Solution and Objectives

- Provide an interactive dashboard to explore synthetic academic records.
- Use a coordinated set of specialized agents to organize, analyze, review, and communicate results.
- Apply deterministic Python rules for the implemented risk assessment.
- Validate workflow outputs and present actionable suggestions for faculty consideration.
- Keep follow-up communications as drafts requiring human review.

4. System Architecture

The Streamlit interface is the user-facing layer. It uses the FastAPI backend to invoke the orchestrated workflow. The workflow reads synthetic CSV records, uses Gemini for language-based agent tasks where configured, and applies Python logic for deterministic execution. A validation stage checks the structured result before it is returned to the interface. If Gemini is unavailable, fallback behavior is available in the implementation.

The accompanying Acadex_AI_Architecture.png provides a visual overview. The editable .drawio file can be opened in diagrams.net.

5. Agent Roles and Responsibilities

Agent	Responsibility
Planner Agent	Organizes the workflow and identifies the analysis steps required.

Research Agent	Examines academic indicators and prepares analysis context.
Executor Agent	Uses deterministic Python rules to calculate risk and produce structured results.
Critic Agent	Reviews the result and recommendations for consistency.
Software Development Agent	Checks dataset structure, required fields, and record identifiers.
Business Automation Agent	Prepares actionable intervention suggestions for faculty review.

6. Data Flow and Orchestration

1. Synthetic student records are loaded from the CSV dataset.
2. The user explores the records through the Streamlit dashboard and selects a student for workflow analysis.
3. The interface sends a request to the FastAPI backend.
4. The backend orchestrates agent tasks, using Gemini for supported language-based tasks and Python for deterministic calculations.
5. Validation checks the result before the API returns it.
6. The dashboard displays the risk assessment and suggested next step for faculty review.

7. Proof-of-Concept Results

In a tested example using synthetic record ST1002, the workflow returned a Moderate risk classification. The example record reported attendance of 69.9%, a current average of 67.9, a previous score of 54.7, a positive score change of 13.2, and 7 of 10 assignments completed. The suggested action was a faculty check-in during the current week. These values illustrate prototype behavior and are not real student data or validated predictions.

The dashboard contains 500 synthetic student records. A backend test through the FastAPI Swagger interface confirmed Gemini-supported execution across four agents in the tested workflow and a successful validation result. This is a functional demonstration, not evidence of real-world academic prediction accuracy.

8. Validation and Testing

- Backend endpoint exercised through FastAPI Swagger UI.
- Gemini provider reported as configured in the successful test.
- Workflow returned a structured result and validation_passed = true.
- Dashboard displayed aggregate statistics and supported filtering and student search.

9. Technology Stack

- Python for application logic and deterministic risk rules.
- Streamlit for the interactive dashboard.
- FastAPI for the backend API and Swagger documentation.

- Google Gemini for supported language-based agent tasks.
- Pandas / CSV for synthetic data handling.

10. Limitations
- The prototype uses synthetic records and has not been evaluated on real institutional data.
- Risk classifications depend on the rules and indicators currently implemented.
- Language-model outputs can be incorrect, inconsistent, or sensitive to prompt and model changes.
- The current proof of concept does not establish causal relationships between indicators and academic outcomes.
- Intervention suggestions must be reviewed by authorized faculty; the system must not make consequential academic decisions autonomously.
- Privacy, fairness, access control, auditability, and institutional integration require further work before deployment.

11. Responsible AI and Human Oversight

Acadex AI is intended to support, not replace, faculty judgment. Any suggested outreach is a draft and should be reviewed by an authorized person who can consider context not represented in the dataset. A real deployment would require appropriate consent and governance, secure data handling, fairness evaluation, transparent criteria, and a process for students and staff to correct inaccurate information.

12. Future Scope
- Integrate with approved institutional learning management systems.
- Evaluate on consented, anonymized, representative data with documented metrics.
- Add explainability for each risk factor and allow faculty feedback.
- Track intervention outcomes and evaluate whether recommendations help.
- Add authentication, role-based access, audit logs, and stronger data governance.
- Improve testing, observability, and deployment reliability.

13. How to Run the Prototype

Install the project dependencies from the supplied requirements files. Configure the Gemini API key in a local .env file (never commit this file). Start the FastAPI backend from the repository root:

uvicorn backend.main:app --reload

Open http://127.0.0.1:8000/docs to inspect the API. In a separate terminal, run:

streamlit run streamlit_app.py

Open the local Streamlit URL shown in the terminal, usually http://localhost:8501. Confirm the file paths and dependencies match the uploaded repository before running.

14. Conclusion

Acadex AI demonstrates a possible multi-agent workflow for academic monitoring: data exploration, agent orchestration, rule-based risk assessment, output validation, and faculty-reviewed recommendations. The prototype provides a foundation for further testing and responsible development, while its synthetic-data basis and limited validation mean it should not be used for real student decisions without substantial evaluation and safeguards.